实验 5. 分支限界法的应用-布线等问题

实验内容

本实验要求基于算法设计与分析的一般过程（即待求解问题的描述、算法设计、算法描述、算法正确性证明、算法分析、算法实现与测试），通过分支限界法（包括队列式和优先队列式）的在实际问题求解实践中，加深理解其基本原理和思想以及求解步骤。求解的问题为布线问题。**作为挑战：可以考虑分支限界法在其他问题（如旅行商问题、0-1 背包问题）。**

实验目的

- 理解分支界限法的核心思想以及求解过程；
- 从算法分析与设计的角度，对方块布线等问题的基于分支界限法求解有进一步的理解。

环境要求

无特别要求。算法实现可以自由选择 C, C++, Java, Python 等。

实验步骤

步骤 1：理解问题，给出问题的描述。

步骤 2：算法设计，包括算法策略与数据结构的选择；

步骤 3：描述算法。采用源代码以外的形式（如**伪代码**、流程图等）来描述；

步骤 4：算法的正确性证明。需要这个环节，在理解的基础上对算法的正确性给予证明（这里可以略去。有兴趣可以深入一下“多米诺性质”）；

步骤 5：算法复杂性分析，包括时间复杂性和空间复杂性（关于平均情况下的时间复杂性分析方法，有兴趣可以深入一下“蒙特卡洛方法”）；

步骤 6：算法实现与测试。附上代码或以附件的形式提交，同时贴上算法运行结果截图；

步骤 7：技术上、分析过程中等各种心得体会与备忘，需要言之有物。

说明：步骤 1-6 在“实验结果”一节中描述，步骤 7 在“实验总结”一节中描述。

实验结果

1、 理解问题，给出问题的描述。

在 N×M 的方格阵列中，指定一个方格的中点为 a，另一个方格的中点为 b 布线问题要求找出 a 到 b 的最短布线方案（即最短路径）

2、 算法设计，包括算法策略与数据结构的选择；

维护一个二维数组存储图信息，0 代表墙，1 代表通路，2 代表起点，3 代表终点，再维护一个二维数组存储图上每个结点的访问情况，防止在进行广度优先搜索遍历时重复访问一个结点。

从起点开始，进行广度优先搜索，维护一个队列，先将起点放入队列中，查看

当前位置的上下左右是否可走（不是墙、未出界且未访问过），可走则将其放入队列中，循环遍历队列中的每个结点，直至队列为空

3、 描述算法。希望采用源代码以外的形式，如**伪代码**、流程图等；

```
void bfs()
    visit[start.x, start.y] = 1
    start.path.append(start.x, start.y)
    while !qe.empty()
        here=qe.front()
        qe.pop()
        for i from 0 to 3
            next = here + op[i]
            next.path.append(next.x, next.y)
            if(next.x == final.x && next.y == final.y)
                res = next
            if(G[next.x, next.y] && !visit[next.x, next.y])
                visit[next.x, next.y] = 1
                qe.push(next)
```

4、 算法的正确性证明。需要这个环节，在理解的基础上对算法的正确性给予证明；

从起点开始进行 bfs，层层递进，当首次搜索到终点时即为最短路径

5、 算法复杂性分析，包括时间复杂性和空间复杂性；

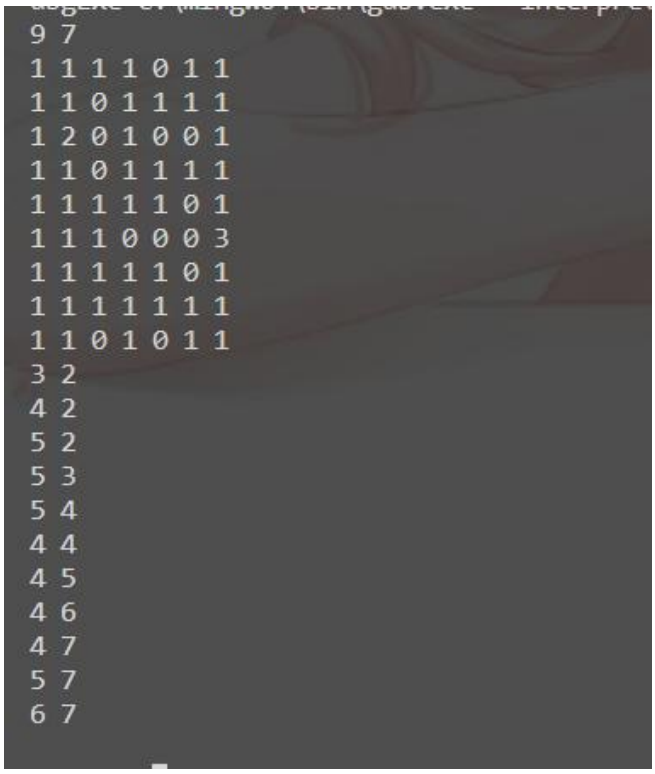
时间复杂度：约束函数耗时为$O(4^n)$, 限界函数耗时最坏为$O(1)$

故时间复杂度为$O(4^n)$（n 为最短路径的长度）

6、 算法实现与测试。附上代码或以附件的形式提交，同时贴上算法运行结果截图；

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

#define PII pair<int,int>
#define MKPII(a,b) make_pair(a,b)

const int N = 1e3 + 10;
int G[N][N];
int visit[N][N];
int n, m, num;
typedef struct {
    int x, y;
    vector<pair<int,int>> path;
} node;
node Start, Final, here, Next, res;
pair<int, int> op[4] = {pair<int, int>(1, 0), pair<int, int>(-1, 0),
                        pair<int, int>(0, 1), pair<int, int>(0, -1)};

void bfs() {
```

```
    queue<node> qe;
    visit[Start.x][Start.y] = 1;
    Start.path.push_back(MKPII(Start.x, Start.y));
    qe.push(Start);
    while (!qe.empty()) {
        here = qe.front();
        qe.pop();
        for (int i = 0; i < 4; i++) {
            Next = here;
            Next.x = here.x + op[i].first;
            Next.y = here.y + op[i].second;
            Next.path.push_back(MKPII(Next.x, Next.y));
            if (Next.x == Final.x && Next.y == Final.y) {
                res = Next;
                return;
            }

            if (G[Next.x][Next.y] && !visit[Next.x][Next.y]) {
                visit[Next.x][Next.y] = 1;
                qe.push(Next);
            }
        }
    }
}

int main() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> num;
            G[i][j] = num;
            if (num == 2)
                Start.x = i, Start.y = j;
            else if (num == 3)
                Final.x = i, Final.y = j;
        }
    }
    bfs();
    for (int i = 0;i < (int)res.path.size();i++){
        cout << res.path[i].first << " " << res.path[i].second << endl;
    }
    return 0;
}
```



```
9 7
1 1 1 1 0 1 1
1 1 0 1 1 1 1
1 2 0 1 0 0 1
1 1 0 1 1 1 1
1 1 1 1 1 0 1
1 1 1 0 0 0 3
1 1 1 1 1 0 1
1 1 1 1 1 1 1
1 1 0 1 0 1 1
3 2
4 2
5 2
5 3
5 4
4 4
4 5
4 6
4 7
5 7
6 7
```

实验总结

本次实验遇到的问题及其解决方法（如果有的话），以及心得体会等的总结。需言之有物。

通过本次实验，我对广度优先算法有了更深一步的理解，在面对分支界限类问题时，可以采取广度优先算法，并结合界限函数，能较容易地得到问题的解